

# Graduate Algorithms

## MidSem Exam: Solutions

Vardhan Kumar Ray

September 21, 2026

### Note

- These are my own solutions are **not** official solutions.
- Most of these problems admit several correct solutions
- Mistakes are mine. Please tell me if you find one.
- Throughout,  $\log$  means  $\log_2$ .

### Question 1: Sorting under Structure

|                    |
|--------------------|
| <i>3+3+4 marks</i> |
|--------------------|

Let  $A[1 \dots n]$  hold distinct integers and let  $1 \leq k < n$ .

#### (a) $O(nk)$ time, $O(1)$ extra space

The first  $n - k$  entries are already sorted. Insert the last  $k$  entries into that sorted part one at a time, exactly as insertion sort does.

---

```
1: for  $i \leftarrow n - k + 1$  to  $n$  do
2:    $key \leftarrow A[i]$ ,  $j \leftarrow i - 1$ 
3:   while  $j \geq 1$  and  $A[j] > key$  do
4:      $A[j + 1] \leftarrow A[j]$ ,  $j \leftarrow j - 1$ 
5:   end while
6:    $A[j + 1] \leftarrow key$ 
7: end for
```

---

The invariant is that before the iteration for  $i$ , the prefix  $A[1 \dots i - 1]$  is sorted. It holds at the start because  $A[1 \dots n - k]$  is given sorted. The body shifts every entry larger than  $key$  one place right and drops  $key$  into the gap, so  $A[1 \dots i]$  is sorted afterwards. After the last iteration the whole array is sorted.

Each of the  $k$  insertions shifts at most  $n - 1$  entries, so the time is  $O(nk)$ . Only the variables  $i, j, key$  are used, so the extra space is  $O(1)$ .

#### (b) $O(n + k \log k)$ time, $O(n)$ extra space

Sort the tail, then merge it with the head.

1. Sort  $A[n - k + 1 \dots n]$  with merge sort:  $O(k \log k)$ .
2. Merge the two sorted runs  $A[1 \dots n - k]$  and  $A[n - k + 1 \dots n]$  into an auxiliary array  $B[1 \dots n]$ :  $O(n)$ .
3. Copy  $B$  back into  $A$ :  $O(n)$ .

Step 1 leaves both runs sorted and merging two sorted runs produces their sorted union, which is all of  $A$ .  $O(k \log k) + O(n) + O(n) = O(n + k \log k)$  time. The auxiliary array is  $O(n)$  extra space.

#### (c) Every entry at most $k$ places from home: $O(n \log(k + 1))$ time, $O(k)$ extra space

Keep a min-heap holding a sliding window of  $k + 1$  entries.

---

```

1:  $H \leftarrow$  min-heap built from  $A[1 \dots \min(k+1, n)]$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $x \leftarrow \text{EXTRACTMIN}(H)$ 
4:   if  $i + k + 1 \leq n$  then
5:      $\text{INSERT}(H, A[i + k + 1])$ 
6:   end if
7:    $A[i] \leftarrow x$ 
8: end for

```

---

We show by induction that the  $i$ -th iteration outputs the  $i$ -th smallest entry. Let  $y$  be the  $i$ -th smallest. It is at position  $i$ , so by hypothesis its original position  $p$  satisfies  $|p - i| \leq k$ , hence  $p \leq i + k$ . Every entry with original position at most  $i + k$  has been inserted into  $H$  by the start of iteration  $i$ , and by induction the  $i - 1$  entries removed so far are exactly the  $i - 1$  smallest, none of which is  $y$ . So  $y$  is in  $H$ , and being the smallest entry not yet removed, it is the minimum of  $H$ .

The heap never holds more than  $k + 1$  entries, so each  $\text{EXTRACTMIN}$  and  $\text{INSERT}$  costs  $O(\log(k + 1))$ . There are  $n$  of the former and at most  $n$  of the latter, giving  $O(n \log(k + 1))$  time and  $O(k)$  extra space.

## Question 2: Counting Three-Term Progressions

4+3+3 marks

(a)

Multiplying out,

$$P(x)^2 = \left( \sum_{a \in S} x^a \right) \left( \sum_{c \in S} x^c \right) = \sum_{a \in S} \sum_{c \in S} x^{a+c},$$

so the coefficient of  $x^t$  is the number of *ordered* pairs  $(a, c) \in S \times S$  with  $a + c = t$ .

**Order matters** because  $(a, c)$  and  $(c, a)$  are counted separately whenever  $a \neq c$ . **An element may be paired with itself**, if  $t$  is even and  $t/2 \in S$ , the pair  $(t/2, t/2)$  contributes 1.

(b)

Fix  $b \in S$  and let  $N_b = [x^{2b}]P(x)^2$ , the number of ordered pairs  $(a, c) \in S \times S$  with  $a + c = 2b$ . Then

$$\#\{\text{three-term progressions with middle element } b\} = \frac{N_b - 1}{2}.$$

A three-term progression with middle element  $b$  is a pair  $a < c$  in  $S$  with  $a + c = 2b$ . Among the  $N_b$  ordered pairs counted by  $N_b$ , exactly one has  $a = c$ , namely  $(b, b)$ , which is present because  $b \in S$ . The remaining  $N_b - 1$  ordered pairs have  $a \neq c$  and fall into two-element groups  $\{(a, c), (c, a)\}$ . Hence the count is  $(N_b - 1)/2$ , and in particular  $N_b$  is always odd.

(c)

1. Build the coefficient vector of  $P(x)$ , which has degree at most  $M$ : set the  $a$ -th coefficient to 1 for each  $a \in S$  and to 0 otherwise.
2. Compute  $P(x)^2$  by one FFT-based multiplication.
3. Return  $\sum_{b \in S} \frac{[x^{2b}]P(x)^2 - 1}{2}$ .

By (b) the  $b$ -th term counts exactly the progressions whose middle element is  $b$ . Every three-term progression has exactly one middle element, which lies in  $S$ , so the sum counts each progression exactly once. All indices  $2b$  are at most  $2M$ , which is within the degree of  $P(x)^2$ .

Step 1 is  $O(M)$ . Step 2 is  $O(M \log M)$  by the assumption on FFT. Step 3 means at most  $|S| \leq M + 1$  coefficients, so it is  $O(M)$ . The total is  $O(M \log M)$ .

**Question 3: Second Smallest Key in a Binomial Heap****7+3 marks****(a)**

Let  $r$  be the root of minimum key, so  $r$  holds the smallest key in the heap. The second smallest key is either at another root or at a child of  $r$ . Let  $x$  be the node holding the second smallest key. If  $x$  has a parent  $p$ , then heap order gives  $\text{key}(p) < \text{key}(x)$ , and the only key smaller than  $x$ 's is the minimum, so  $p = r$  and  $x$  is a child of  $r$ . If  $x$  has no parent, it is a root.

**Algorithm:** Scan the root list and record the two smallest root keys. Let  $r$  be the root attaining the smallest. Then scan the children of  $r$  and take the smallest key among them. Return the smaller of the second smallest root key and that value.

A binomial heap on  $n$  elements has at most  $\lfloor \log n \rfloor + 1$  trees, so the root list has  $O(\log n)$  entries. The root of a binomial tree  $B_j$  has exactly  $j$  children, and  $j \leq \log n$ , so  $r$  has  $O(\log n)$  children. Both scans are  $O(\log n)$ , and the total is  $O(\log n)$  in the worst case.

**(b)**

In a lazy heap, insertions and melds only concatenate root lists, so the number of roots  $t$  can be as large as  $n$ , and part (a) no longer runs in  $O(\log n)$ .

**Algorithm:**

1. *Consolidate.* Keep an array  $D[0 \dots \lfloor \log n \rfloor]$  of roots indexed by degree. Walk the root list, for each root  $u$ , while  $D[\deg u]$  holds some root  $w$ , link  $u$  and  $w$  (the one with the larger key becomes a child of the other), clear  $D[\deg u]$ , and continue with the merged tree, whose degree is one higher. Otherwise store  $u$  in  $D[\deg u]$ . The surviving entries of  $D$  form the new root list.
2. *Search.* Run the procedure of part (a) on the consolidated heap.

Linking only rearranges trees, so every element and every key is preserved. After step 1 there is at most one tree per degree, so the heap is a proper binomial heap with at most  $\lfloor \log n \rfloor + 1$  roots, and step 2 costs  $O(\log n)$  in the worst case.

Take the potential  $\Phi$  = number of roots. Consolidating a list of  $t$  roots performs at most  $t - 1$  links and touches each root a constant number of times, so its actual cost is  $O(t + \log n)$ , and it leaves at most  $\lfloor \log n \rfloor + 1$  roots. Hence, amortised cost

$$= O(t + \log n) + (\Phi_{\text{after}} - \Phi_{\text{before}}) = O(t + \log n) + (O(\log n) - t) = O(\log n),$$

and step 2 adds only  $O(\log n)$  worst case. So the whole operation is amortised  $O(\log n)$ .

**Question 4: Scheduling at Most  $k$  Intervals****10 marks**

We are given intervals  $(s_i, f_i, v_i)$  for  $i = 1, \dots, n$  and an integer  $k < n$ , and want a set of pairwise disjoint intervals, of size at most  $k$ , whose total value is largest.

Sort the intervals by finishing time, so that  $f_1 \leq f_2 \leq \dots \leq f_n$ . For each  $i$  let

$$p(i) = \max\{j < i : f_j \leq s_i\},$$

with  $p(i) = 0$  if no such  $j$  exists: the last interval that finishes no later than interval  $i$  begins. Since the  $f_j$  are sorted, each  $p(i)$  is one binary search.

Let  $\text{OPT}(i, t)$  be the largest value obtainable from intervals  $1, \dots, i$  using at most  $t$  of them. Then  $\text{OPT}(0, t) = \text{OPT}(i, 0) = 0$ , and for  $i, t \geq 1$

$$\text{OPT}(i, t) = \max\left\{\text{OPT}(i-1, t), v_i + \text{OPT}(p(i), t-1)\right\}.$$

The answer is  $\text{OPT}(n, k)$ .

Consider an optimal solution for  $(i, t)$ . Either it omits interval  $i$ , in which case it is an optimal solution for  $(i-1, t)$  and the first branch is right or it uses interval  $i$ , in which case every other interval it uses

finishes at or before  $s_i$  and so has index at most  $p(i)$ , and it uses at most  $t - 1$  of them, so the rest is an optimal solution for  $(p(i), t - 1)$  and the second branch is right. Both branches are achievable, so the maximum of the two is the optimum.

**Recovering the schedule:** Start at  $(i, t) = (n, k)$ . If  $\text{OPT}(i, t) = \text{OPT}(i - 1, t)$ , move to  $(i - 1, t)$ , otherwise report interval  $i$  and move to  $(p(i), t - 1)$ . Stop when  $i = 0$  or  $t = 0$ . This walks back through at most  $n$  states, so it costs  $O(n)$ .

Sorting is  $O(n \log n)$  and the  $n$  binary searches are  $O(n \log n)$ . The table has  $n \cdot k$  entries, each computed in  $O(1)$  from two earlier entries, so filling it is  $O(nk)$ . Recovery takes  $O(n)$ . The total is  $O(n \log n + nk)$ .

### Question 5: Farthest Pair of Distinct Colours

2+4+4 marks

We are given  $n$  points in the plane, point  $i$  coloured  $c_i$ , and an algorithm **A** that, for two colours, returns in  $O(n \log n)$  time the largest distance between two points of different colour. Here,  $\text{ans}$  = the largest distance between two points of different colour, over  $k$  colours.

**(a)**  $O(nk^2 \log n)$

For every pair of colours  $\{a, b\}$  with  $a \neq b$ , run **A** on the points coloured  $a$  or  $b$ , and return the largest value obtained.

Every value returned is the distance between two points of different colours, so the answer is at most  $\text{ans}$ . Conversely the pair realising  $\text{ans}$  has two colours  $a \neq b$ , and it is present in the call for  $\{a, b\}$ , so that call returns at least  $\text{ans}$ .

There are  $\binom{k}{2} = O(k^2)$  pairs and each call costs  $O(n \log n)$ , giving  $O(nk^2 \log n)$ .

**(b)**  $O(n \log n \log k)$

Use the bits of the colour index. Write each colour in  $\{1, \dots, k\}$  in binary, using  $\lceil \log k \rceil$  bits. For each bit position  $t = 0, 1, \dots, \lceil \log k \rceil - 1$ , two-colour the points by the  $t$ -th bit of  $c_i - 1$ , run **A** on this two-colouring, and return the largest value obtained. Skip a bit position if it puts all points on one side.

In any one of these calls the two groups consist of points whose colour indices differ in bit  $t$ , so any pair across the groups has different colours. Hence no call can return more than  $\text{ans}$ . Conversely, let  $i, j$  realise  $\text{ans}$ . Since  $c_i \neq c_j$ , their indices differ in some bit  $t$ , so in the call for that bit they lie in different groups and the call returns at least their distance.

There are  $\lceil \log k \rceil$  calls, each  $O(n \log n)$ , so  $O(n \log n \log k)$ .

**(c)**

Now each point carries an object  $c_i$  from a set of  $k$  distinct objects, and the only operation available is the test  $c_i < c_j$  in  $O(1)$  time.

1. Sort the  $n$  points by their colour object, using the comparison  $c_i < c_j$ . This costs  $O(n \log n)$  comparisons.
2. Scan the sorted list, keeping a counter that starts at 1 and increases by 1 whenever the colour object changes. Two neighbouring objects are equal exactly when neither  $c_i < c_j$  nor  $c_j < c_i$  holds, which is two  $O(1)$  tests. Given each point the current counter value as its integer colour. This costs  $O(n)$ .
3. Run the algorithm of part (b) on these integer colours.

The objects are distinct and totally ordered by  $<$ , so step 2 assigns equal integers to points of equal colour and different integers to points of different colour, and the integers used are exactly  $1, \dots, k$ . Relabelling in this way changes no pair's status as same-coloured or differently coloured, so part (b) applied to the labels returns  $\text{ans}$ .

$O(n \log n) + O(n) + O(n \log n \log k) = O(n \log n \log k)$ .